Git Tutorial: A Step-by-Step Guide

Introduction

Git is a distributed version control system designed to handle everything from small to very large projects with speed and efficiency. It allows multiple developers to work on a project simultaneously without interfering with each other's work. This guide will walk you through the basics of Git, from installation to advanced features.

Table of Contents

Understanding Version Control

Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later. There are two main types of version control systems:

- **Centralized Version Control Systems (CVCS)**: A single server stores all the versions of the project files, and clients check out files from this central place.
- **Distributed Version Control Systems (DVCS)**: Each client mirrors the entire repository. This means that if any server dies, any of the client repositories can be copied back up to the server to restore it.

Git is a DVCS, which means every developer has a complete history of the project on their local machine.

Installing Git

On Windows

1. Download Git from Git for Windows.
2. Run the installer and follow the instructions. Choose the default options unless you have a specific reason to change them.

On macOS

You can install Git using Homebrew:

```
brew install git
```

Alternatively, you can install it as part of Xcode Command Line Tools:

```
xcode-select --install
```

On Linux

For Debian/Ubuntu-based distributions:

```
sudo apt-get update
sudo apt-get install git
```

For Fedora:

```
sudo dnf install git
```

To verify the installation, run:

```
git --version
```

Getting Started with Git

Setting Up Your Identity

After installing Git, the first thing you should do is set up your username and email address. Git uses this information to label the commits you make.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Initializing a Repository

A Git repository is a directory that contains your project files and the history of all changes made to those files.

To create a new repository:

```
mkdir my_project
cd my_project
git init
```

This command creates a `.git` directory inside `my_project` that tracks changes.

Basic Git Commands

`git init`

This command initializes a new Git repository.

```
git init
```

`git clone`

To clone an existing repository:

```
git clone https://github.com/user/repo.git
```

This command copies the repository from GitHub (or any other remote) to your local machine.

`git status`

To check the status of your files in the working directory and staging area:

```
git status
```

`git add`

To add a file to the staging area (preparing it for commit):

```
git add file_name
```

To add all changes:

```
git add .
```

`git commit`

To commit the changes in the staging area:

```
git commit -m "Commit message"
```

`git push`

To push your changes to a remote repository:

```
git push origin branch_name
```

`git pull`

To fetch and merge changes from the remote repository:

```
git pull origin branch_name
```

Branching and Merging

Creating a Branch

To create a new branch:

```
git branch branch_name
```

Switching Branches

To switch to a different branch:

```
git checkout branch_name
```

Merging Branches

To merge a branch into your current branch:

```
git merge branch_name
```

Collaborating with Others

When working with others, it's common to create branches for new features or bug fixes. Here's how you can collaborate:

1. **Create a new branch**: `git checkout -b feature_branch`
2. **Work on your changes** and commit them.
3. **Push the branch**: `git push origin feature_branch`
4. **Create a Pull Request** on GitHub for code review and merging.

Rebasing

Rebasing is the process of moving or combining a sequence of commits to a new base commit. This can be useful for keeping a clean project history.

```
git rebase branch_name
```

Undoing Changes

Undo a Commit

To undo the last commit (but keep the changes):

```
git reset --soft HEAD~1
```

To undo the last commit and discard the changes:

```
git reset --hard HEAD~1
```

Discard Changes in a File

To discard changes in a specific file:

```
git checkout -- file_name
```

Conclusion

This tutorial has covered the basics of Git, from installation to more advanced topics like branching, merging, and rebasing. By mastering these commands and concepts, you'll be well on your way to managing your projects efficiently with Git.

For more detailed information, consult the official Git documentation.
